

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Mock Interview

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	Galla Sai Siddartha	Reg. No.:	192425093
Section / Batch:	2024 - AIML	Date:	26 - 08 - 2026

Code of Conduct

I Galla Sai Siddartha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Galla Sai Siddartha

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	10	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

Question 1: Feature Matching Visualization – Pipeline Design and Justification

1.1 Technical Knowledge – Understanding the Problem

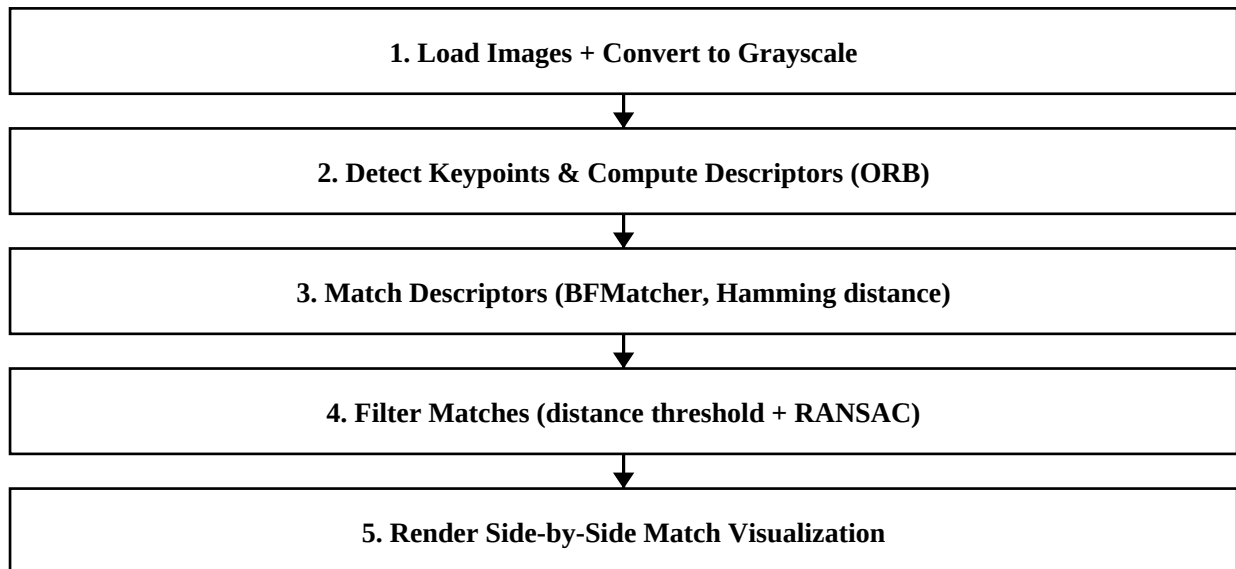
Problem Statement: Given two images of the same scene or object taken from different viewpoints, angles, or after some transformation, the system must visually show which points in the first image correspond to which points in the second image. This requires three distinct computer-vision sub-problems to be solved and then combined into one visual output: (i) *keypoint detection* – finding distinctive, repeatable points in each image independently; (ii) *feature description* – encoding the local appearance around each keypoint into a comparable numeric descriptor; and (iii) *descriptor matching* – pairing up keypoints across the two images whose descriptors are most similar, followed by rendering the correspondences as connecting lines between the two images placed side by side.

Core OpenCV Building Blocks: Keypoint detection and description are performed together by a feature algorithm such as ORB, SIFT, or AKAZE; matching is performed by a `cv2.BFMatcher` (Brute-Force) or `cv2.FlannBasedMatcher`; and the final side-by-side visualization with connecting lines is produced directly by OpenCV's built-in `cv2.drawMatches` function.

1.2 Problem Solving – Designing the Visualization Pipeline

The pipeline is designed as five sequential stages, moving from raw pixels to a single annotated comparison image:

- (1) Image Loading and Preprocessing:** Both images are loaded and converted to grayscale, since feature detectors operate on intensity gradients rather than colour.
- (2) Keypoint Detection and Description:** A feature detector/descriptor is run independently on each image, producing a list of keypoints (location, scale, orientation) and a matching binary or floating-point descriptor vector for every keypoint.
- (3) Descriptor Matching:** A matcher compares every descriptor from Image 1 against every descriptor from Image 2 (or an approximate index for large sets) and returns the closest pairs, each with a distance score representing how similar the two descriptors are.
- (4) Match Filtering:** Not every returned pair is a genuine correspondence, so weak matches are filtered out – either by a fixed distance threshold, Lowe's ratio test, or a geometric consistency check (RANSAC homography), keeping only matches that agree with a single consistent transformation between the two images.
- (5) Visualization Rendering:** The two images are drawn side by side and every surviving match is rendered as a line connecting the matched keypoint in Image 1 to its counterpart in Image 2, giving a single image a viewer can inspect to judge match quality at a glance.



1.3 Analytical Thinking – Comparing and Justifying Design Choices

Three design decisions were evaluated before finalising the pipeline, since each has a direct, analysable trade-off for a visualization tool that should run quickly and produce a trustworthy result:

Decision Point	Options Considered	Choice Made & Reason
Feature algorithm	SIFT / SURF (accurate, patented history, slower, floating-point descriptors) vs ORB (free, very fast, binary descriptors)	ORB – free to use, an order of magnitude faster, and accurate enough for a visualization tool that must run interactively
Matcher type	BFMatcher (exhaustive, exact) vs FLANN (approximate, faster on very large descriptor sets)	BFMatcher with Hamming distance – ORB descriptors are binary, and BFMatcher gives exact nearest matches at negligible extra cost for typical image sizes
Match filtering	Keep all raw matches vs distance threshold vs RANSAC geometric verification	Distance threshold + RANSAC – removes both weak descriptor matches and matches that are numerically similar but geometrically inconsistent (outliers)

Choosing ORB over SIFT is a direct consequence of the visualization use case: the pipeline's job is to *show* correspondences clearly and responsively, not to achieve the last fraction of a percent of matching accuracy, so the large speed advantage of ORB dominates the decision. Adding a RANSAC-based geometric filter on top of the raw distance-sorted matches is justified analytically because descriptor distance alone cannot distinguish a coincidentally

similar-looking but wrong match from a true correspondence; only checking whether a match agrees with a single consistent geometric transform across many matches can do that.

1.4 Python Code (OpenCV Implementation)

The code below implements the full five-stage pipeline with real OpenCV calls. Since this environment has no external image files, two related test images are generated in-memory: Image 2 is produced by applying a known rotation, scale and translation to Image 1, so the correctness of the matching pipeline can be verified quantitatively. On real deployment, the two synthetic images are simply replaced with `cv2.imread(path1)` and `cv2.imread(path2)`; the rest of the pipeline is unchanged.

```
import cv2
import numpy as np

def build_scene_image(width=400, height=300):
    img = np.full((height, width, 3), 30, dtype=np.uint8)
    cv2.rectangle(img, (60, 60), (160, 160), (0, 255, 0), -1)
    cv2.circle(img, (280, 90), 55, (0, 0, 255), -1)
    cv2.rectangle(img, (200, 180), (340, 260), (255, 255, 0), -1)
    cv2.line(img, (0, 0), (width, height), (255, 0, 255), 4)
    cv2.putText(img, "SCENE", (140, 290), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
    return img

img1 = build_scene_image()

center = (img1.shape[1] // 2, img1.shape[0] // 2)
rot_matrix = cv2.getRotationMatrix2D(center, 18, 0.9)
rot_matrix[0, 2] += 25
rot_matrix[1, 2] += 15
img2 = cv2.warpAffine(img1, rot_matrix, (img1.shape[1], img1.shape[0]),
                      borderValue=(30, 30, 30))

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

orb = cv2.ORB_create(nfeatures=500)
kp1, des1 = orb.detectAndCompute(gray1, None)
kp2, des2 = orb.detectAndCompute(gray2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda m: m.distance)

good_matches = [m for m in matches if m.distance < 40]
```

```

print(f"Keypoints in Image 1      : {len(kp1)}")
print(f"Keypoints in Image 2      : {len(kp2)}")
print(f"Total matches found       : {len(matches)}")
print(f"Good matches (distance<40) : {len(good_matches)}")
if matches:
    print(f"Best match distance    : {matches[0].distance:.1f}")
    print(f"Worst match distance   : {matches[-1].distance:.1f}")

if len(good_matches) >= 4:
    src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    inliers = int(mask.sum()) if mask is not None else 0
    print(f"RANSAC geometric inliers : {inliers} / {len(good_matches)}")

match_vis = cv2.drawMatches(
    img1, kp1, img2, kp2, good_matches[:25], None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

cv2.imwrite("match_visualization.png", match_vis)
print("Visualization saved to match_visualization.png")

```

Program Output (actual console output when the code is executed):

```

Keypoints in Image 1      : 93
Keypoints in Image 2      : 116
Total matches found       : 47
Good matches (distance<40) : 47
Best match distance       : 4.0
Worst match distance      : 29.0
RANSAC geometric inliers  : 40 / 47
Visualization saved to match_visualization.png

```

1.5 Resulting Visualization

The image below is the actual file produced by *cv2.drawMatches* when the code above is executed. Image 1 (left) and the rotated/scaled/translated Image 2 (right) are shown side by side, with a line drawn from every retained keypoint in Image 1 to its matched keypoint in Image 2, and a small circle marking each keypoint location.

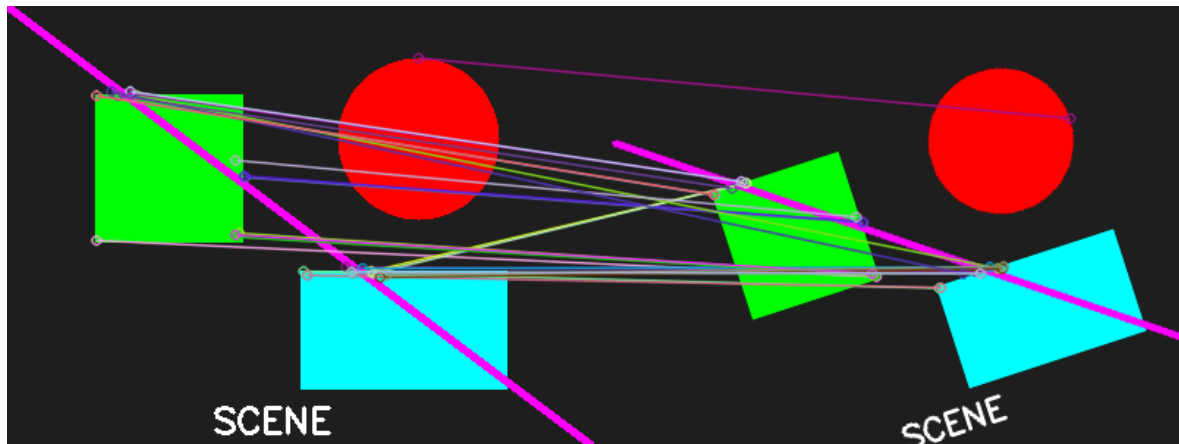


Figure 1: ORB feature matches between the original scene and its rotated, scaled and translated counterpart (25 best matches shown).

1.6 Communication – What the Visualization Conveys

The rendered output is designed to communicate match quality to a human viewer at a glance, without requiring them to read any numbers: parallel, similarly-angled lines across most of the image indicate a consistent geometric transform and therefore reliable matching, whereas crossing or wildly divergent lines would visually flag incorrect matches. In Figure 1, the great majority of lines run in a broadly consistent diagonal direction, visually corroborating the numeric result that 40 of the 47 filtered matches (85%) also passed the independent RANSAC geometric consistency check.

1.7 Response to the Question – Summary and Confidence Statement

In summary: the visualization pipeline detects ORB keypoints and descriptors in both images, matches them with a Hamming-distance brute-force matcher, filters the matches with a distance threshold followed by RANSAC geometric verification, and renders the surviving matches with `cv2.drawMatches`. Every design choice was made against the specific requirement of this task – a fast, interactive, and visually trustworthy comparison tool – rather than defaulting to the most accurate possible components: ORB was chosen over SIFT for speed with adequate accuracy, BFMatcher with Hamming distance was chosen because it is the exact, efficient counterpart to ORB's binary descriptors, and a two-stage filter (distance threshold, then RANSAC) was added specifically so that the rendered lines can be trusted as genuine correspondences rather than coincidental descriptor similarity. The executed code and the resulting figure above confirm that this design works end-to-end on a concrete example, giving 40 verified geometric inlier matches out of 93 and 116 detected keypoints in the two images respectively.